No Man's Land

Design Document
CSEE 4840 – Embedded Systems, Spring 2026

Rohit Biswas (rb3908) Kambinachi Obioha (kno2117) Nicola Paparella (np2953)

Instructor: Prof. Stephen Edwards
April 16, 2026

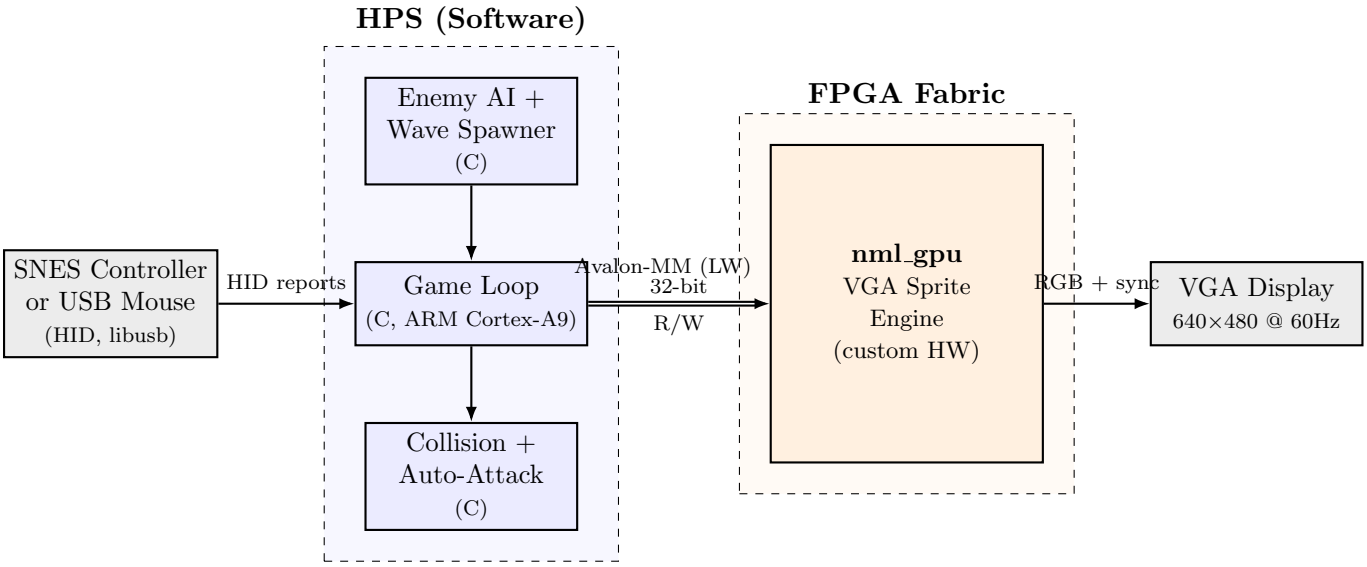
1 Project Summary

No Man's Land is a top-down WWI horde survival game on the DE1-SoC. Player movement is controlled via SNES controller or USB mouse (USB keyboard input is not permitted per professor feedback). Enemy waves spawn from screen edges and chase the player; each surviving wave grants an auto-attack item (mortar, barbed wire, mustard gas, artillery). A custom SystemVerilog peripheral (`nml_gpu`) handles VGA timing, tile-mapped background, and sprite compositing. Game logic, AI, collision, and auto-attack scheduling run in C on the ARM Cortex-A9 (HPS). Software is responsible for all score, health, and wave-number tracking; hardware reads these only to drive the HUD overlay.

The thematic hook (waves shift toward unarmed figures; auto-attacks have no target filter; end-of-run kill breakdown) lives entirely in software wave tables and the end screen. Hardware contract is unchanged.

Design commitment: background scrolling is either fully implemented or fully omitted; the arena is single-screen in the current baseline. This decision is final and will not be revisited in a later phase.

2 Top-Level Block Diagram



The HPS owns all game state (score, health, wave number) and writes sprite lists, background state, and HUD fields to `nml_gpu` each frame over the Avalon-MM lightweight bus. `nml_gpu` owns all video timing and drives the DE1-SoC VGA DAC directly. Input is read via libusb from either the SNES controller or USB mouse.

2.1 Connections in Detail

- **HPS ↔ nml_gpu (Avalon-MM Lightweight Bridge):** 32-bit data, 14-bit byte address window (16 KB region). Single-master (HPS), single-slave (nml_gpu). All transactions are single-cycle reads/writes; no burst, no waitrequest. The peripheral is fully synchronous to the 50 MHz Avalon clock.
- **nml_gpu → VGA DAC:** 24-bit RGB (8-8-8), HSYNC, VSYNC, BLANK, pixel clock (25 MHz, generated by an on-chip PLL inside `nml_gpu`; the Cyclone V cannot produce exactly 25.175 MHz from 50 MHz).
- **SNES Controller or USB Mouse → HPS:** Standard USB 2.0 HID protocol over the HPS USB host port. Detail in Section 3.

3 Input Protocol

Player input uses either the SNES controller (preferred) or a USB mouse. USB keyboard input is not used. Both devices enumerate under Linux on the HPS and are read via libusb interrupt-IN reports.

3.1 SNES Controller Mapping

The SNES controller uses a serial latch/clock/data protocol. A single poll reads 16 bits (one per button). USB-to-SNES adapters expose this as a standard HID gamepad report.

Button	Game Action
D-pad Up / Down / Left / Right	Move N / S / W / E
Right face buttons (X/Y/A/B)	Aim direction (4-way or 8-way via combinations)
B	Manual fire
Start	Menu confirm (item selection, restart)
Select	Pause / quit
L / R	Pick item slot 1/2 on level-up

3.2 USB Mouse Fallback

If the SNES controller is unavailable, a USB mouse provides equivalent input: mouse delta drives movement, left button fires, and right button pauses. Item selection falls back to on-screen prompts.

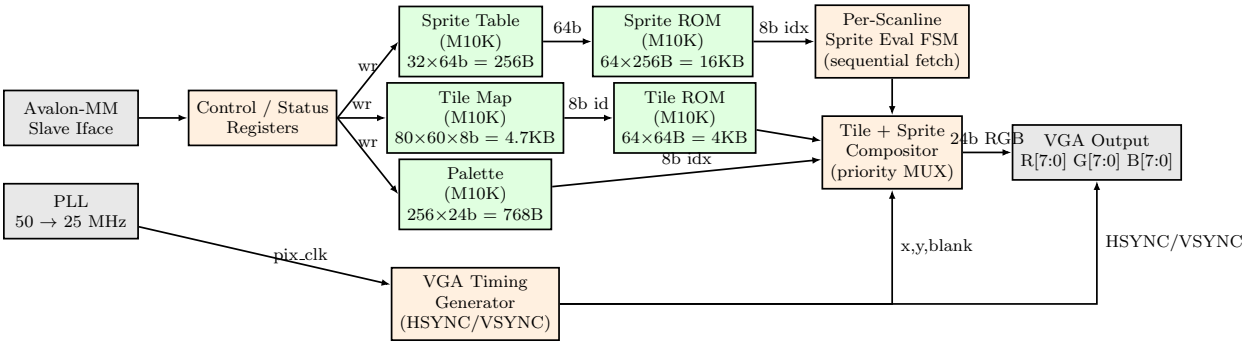
3.3 Input Reader Thread

The libusb reader runs on a pthread with two 16-bit `input_bitmask` buffers and an atomic `active_idx`. Each report decodes into the inactive buffer, then an atomic store flips the index. `input_snapshot()` atomically loads `active_idx` and returns the corresponding buffer. Lock-free, no mutex in the hot path.

4 Custom Peripheral: nml_gpu

nml_gpu accepts writes from the HPS and renders a 640×480@60 Hz frame each refresh by compositing a tile-mapped background with up to 32 sprites.

4.1 Internal Block Diagram



4.2 Memory Inventory

All memories use Cyclone V M10K block RAM (10 Kbit each); total is well within DE1-SoC budget.

Memory	Width	Depth	Total	Access
Sprite Table	64 b	32	256 B	SW write, HW read (sequential eval FSM)
Tile Map	8 b	4800	4.7 KB	SW write, HW read (per pixel)
Palette	24 b	256	768 B	SW write, HW read (per pixel)
Sprite ROM	8 b	16384	16 KB	HW read only (`$readmemh`)
Tile ROM	8 b	4096	4 KB	HW read only (`$readmemh`)
Line Buffer A	24 b	640	1.9 KB	Internal compositor, double buffered
Line Buffer B	24 b	640	1.9 KB	Internal compositor, double buffered
Total			**≈ 29 KB**	

Widths: data paths are 8-bit (palette index, tile ID, sprite pixel) or 24-bit (RGB888). Avalon data is 32-bit. Sprite table entries are 64b packed (Section 5.3); the eval FSM reads one entry per cycle sequentially.

4.3 Rendering Pipeline (Per Pixel)

Line-buffer architecture: one line buffer is filled by the sprite fetch FSM during HBLANK while the other is read out to the VGA DAC. Per scanline:

1. **Timing:** `vga_timing` produces (`x`, `y`, `blank`, `hsync`, `vsync`).
2. **Sprite eval** (during HBLANK preceding scanline N+1): the `sprite_eval` FSM walks the 32-entry sprite table *one entry per cycle*, checking whether each sprite's Y bounding box intersects scanline N+1. A sprite is fetched as soon as it is identified as visible; all 32 checks complete well within the HBLANK budget at 25 MHz. Up to 8 visible sprites are collected in an active buffer, dropping lowest-priority-first if more than 8 qualify.

3. **Sprite fetch** (still during HBLANK): for each sprite in the active buffer, read up to 16 palette-index bytes from Sprite ROM and write non-transparent bytes into the fill buffer at the sprite's on-screen x offset. Higher-priority sprites overwrite lower. Worst case $8 \times 16 = 128$ Sprite-ROM reads against a 160-pixel HBLANK + 640-pixel fill-idle budget.
4. **Visible scanline** N+1, per pixel:
   (a) Tile map lookup: $(x>>3, y>>3) \rightarrow$ tile ID (8b).
   (b) Tile ROM lookup: `tile_id, x[2:0], y[2:0]` $\rightarrow$ palette index (8b).
   (c) Read fill buffer at column x: sprite palette index (0 = transparent).
   (d) Priority MUX: non-transparent sprite pixel wins; otherwise tile pixel.
   (e) Palette lookup: 8b index $\rightarrow$ 24b RGB $\rightarrow$ VGA DAC.
5. At HSYNC the two line buffers swap roles; the new fill buffer is *cleared by writing tile-background pixels* (not zeroes) during its fill phase, so no blank/black flash occurs between sprite draws.

Only one Sprite ROM read per pixel clock is needed, matching the M10K dual-port limit; no ROM replication required.
Handshake: the compositor never stalls (VGA must be deterministic). The sprite table uses VBLANK-based double-buffering: SW writes to the shadow table, sets `CTRL.SWAP=1`, and the swap commits at next VSYNC with `STATUS.VBLANK` pulsing. Shadow memory is *not* used for synchronization; VBLANK is the sole commit trigger. Everything else is fire-and-forget.

4.4 HUD Overlay

Fixed overlay gated by `CTRL.HUD_ON=1`, reads only the Player State registers (Section 5.2). Score, health, and wave number are all maintained in software and written to the Player State mailbox registers; hardware reads these values only to render the HUD. Occupies a reserved 16-pixel strip at the top of the screen ($y \in [0, 15]$):

- **HP bar** – cols 0–191, red/green gradient proportional to `hp`/100.
- **Wave** – cols 256–383, displayed using tile-based font (Tile ROM slots 56–63).
- **Score** – cols 448–639, 6-digit decimal of `SCORE`, same tile font.

HUD is a fourth input to the priority MUX, ranked above all sprites; does not count against the 8-sprite/line cap. Wave spawn logic confines game action to $y \geq 24$ so sprites do not compete with the strip.

5 Register Map

`nml_gpu` occupies a 16 KB window on the HPS-to-FPGA Lightweight bridge. Base offsets:

Region	Offset	Size	Description
Control/Status	`0x0000`	16 B	4 control/status registers (see below)
Player State	`0x0010`	16 B	player x, y, hp, score (SW-maintained; consumed by HUD overlay)

Region	Offset	Size	Description
Sprite Table	0x0100	256 B	32 entries × 8 B
Palette	0x0400	1 KB	256 entries × 4 B (RGB888 + pad)
Tile Map	0x1000	4.7 KB	4800 bytes (80 cols × 60 rows)

5.1 Control / Status Registers

Offset	Name	Width	Access	Behavior
0x00	CTRL	32 b	R/W	bit0 ENABLE (1 = enable video output), bit1 SWAP (write 1 to commit shadow sprite table at next VSYNC; auto-clears), bit2 HUD_ON, bits31:8 reserved
0x04	STATUS	32 b	R	bit0 VBLANK (1 during VBLANK), bit1 SWAP_PENDING, bits15:8 frame counter mod 256
0x08	BG_SCROLL	32 b	R/W	bits15:0 = scroll_x (signed), bits31:16 = scroll_y (signed). Arena is single-screen in baseline; background scrolling is either fully implemented or not at all.
0x0C	IRQ_MASK	32 b	R/W	bit0 enables VBLANK interrupt to HPS (not used in baseline)

5.2 Player State Registers (0x0010–0x001F)

Pure mailbox registers. Software maintains all game state and writes these registers each frame; hardware reads them only to drive the HUD overlay (HP bar, wave number, score). No game logic depends on hardware reading these values.

Offset	Field	Encoding
0x10	PLAYER_POS	bits15:0 = px (pixels), bits31:16 = py (pixels)
0x14	PLAYER_STATS	bits7:0 = hp, bits15:8 = wave, bits31:16 = level
0x18	SCORE	32-bit unsigned score
0x1C	KILL_COUNT	32-bit unsigned kills (HUD only)

5.3 Sprite Table Entry Format

Each sprite entry is 8 bytes (two 32-bit words). Software writes both words; hardware reads them as a single 64-bit value. Player position is encoded here (not in a separate register) so the hardware compositor has a single source of truth for all sprite positions.

Word	Field	Bits	Meaning
W0	`x`	[15:0]	X position (signed pixels, −128 to 767 visible). To hide a sprite, write $x = -256$ (off-screen sentinel).
W0	`y`	[31:16]	Y position (signed pixels). To hide a sprite, write $y = -256$ (off-screen sentinel).
W1	`sprite_id`	[7:0]	Index into Sprite ROM. Only bits [5:0] are meaningful (0–63); bits [7:6] are reserved and must be written 0.
W1	reserved	[9:8]	reserved for future size encoding; must be 0. All sprites are 16×16 in baseline.
W1	`flip`	[11:10]	bit10 = HFlip, bit11 = VFlip
W1	`priority`	[14:12]	0 = highest, 7 = lowest (for per-scanline culling)
W1	`palette_offset`	[23:16]	added to sprite-ROM palette index before lookup (allows team recoloring)
W1	reserved	[31:24]	0

Note: there is no `active` bit. Sprites are hidden by writing an off-screen sentinel coordinate ($x = -256$, $y = -256$). The sprite eval FSM skips any entry whose bounding box does not intersect the visible frame; no explicit enable flag is required, reducing wiring complexity.
Reads return last-written values (not the live shadow). **Writes** land in shadow immediately. Sprite table commits at next VSYNC after `CTRL.SWAP=1` (Section 4.3). Palette and tile map use a per-region 1-bit dirty flag: on any write, the flag sets; at the next `hsync` rising edge, active RAM is updated from shadow and the flag clears. No torn pixels within a scanline. SW can write any region at any time without waiting for VBLANK.

6 Software Architecture (C on HPS)

6.1 SDL Hardware Simulator

Before targeting the FPGA, a software simulation of `nml_gpu` is implemented using SDL2. The simulator accepts the same `ioctl`-style API calls as the hardware driver and renders to an SDL window using sprite and tile ROM data loaded from static C arrays (generated by a script that converts ROM hex files into `const uint8_t[]` declarations). This allows game logic to be developed and tested on a host machine without hardware access and provides a reference implementation to validate the SystemVerilog compositor.

```
/* sdl_sim.h -- drop-in replacement for nml_gpu.h during simulation */
int  nml_open(void);      /* opens SDL window instead of mmap */
void nml_write_sprite(int slot, const nml_sprite_t *s);
void nml_commit_frame(void); /* SDL_RenderPresent instead of VBLANK wait */
```

The ROM conversion script reads `sprite_rom.hex` and `tile_rom.hex` and emits a `.c` file with static arrays, compiled directly into the simulator binary. No file I/O at runtime.

6.2 Source File Organization

File	Purpose
`main.c`	Entry point, mmap of `nml_gpu`, main loop, frame timing
`nml_gpu.h`	Register offsets, sprite struct, low-level write helpers
`nml_gpu.c`	Implementation of writers (`write_sprite`, `commit_frame`, etc.)
`sdl_sim.c`	SDL2 simulator implementing the same API as `nml_gpu.c`
`rom_data.c`	Auto-generated static arrays for sprite and tile ROM
`gen_rom.py`	Script: converts .hex ROM files to `rom_data.c`
`input.h/.c`	libusb reader thread for SNES controller or mouse
`game.h/.c`	Top-level game state machine (menu / playing / level-up / dead)
`entity.h/.c`	Entity pool (player, enemies, projectiles, hazards) and update
`ai.c`	Enemy chase behavior, target selection
`collision.c`	AABB pairwise checks; emits damage events
`autoatk.c`	Auto-attack cooldowns and projectile spawning
`wave.h/.c`	Wave table, spawner, end-of-wave item selection
`render.c`	Translates entity pool into sprite-table writes
`audio.c`	(Stretch) sound effects via Wolfson WM8731

6.3 Header: `nml_gpu.h`

```c
#ifndef NML_GPU_H
#define NML_GPU_H

#include <stdint.h>

/* --- Register offsets (bytes from peripheral base) --- */
#define NML_REG_CTRL          0x0000
#define NML_REG_STATUS        0x0004
#define NML_REG_BG_SCROLL     0x0008
#define NML_REG_IRQ_MASK      0x000C

#define NML_REG_PLAYER_POS    0x0010
#define NML_REG_PLAYER_STATS  0x0014
#define NML_REG_SCORE         0x0018
#define NML_REG_KILL_COUNT    0x001C

#define NML_SPRITE_TABLE_BASE 0x0100   /* 32 entries * 8 B = 256 B */
#define NML_PALETTE_BASE      0x0400   /* 256 entries * 4 B = 1 KB */
#define NML_TILEMAP_BASE      0x1000   /* 80 * 60 = 4800 B */

/* --- CTRL bits --- */
#define NML_CTRL_ENABLE   (1u << 0)
#define NML_CTRL_SWAP     (1u << 1)
#define NML_CTRL_HUD_ON   (1u << 2)

/* --- STATUS bits --- */
#define NML_STATUS_VBLANK         (1u << 0)
#define NML_STATUS_SWAP_PENDING   (1u << 1)

/* Off-screen sentinel: write x=NML_SPRITE_HIDE, y=NML_SPRITE_HIDE to hide */
#define NML_SPRITE_HIDE  ((int16_t)-256)

/* --- Sprite descriptor (matches HW layout) --- */
typedef struct {
```

```
    int16_t  x;           /* NML_SPRITE_HIDE to hide */
    int16_t  y;           /* NML_SPRITE_HIDE to hide */
    uint8_t  sprite_id;
    uint8_t  flags;       /* bits1:0 reserved (size, must be 0),
                             bit2 hflip, bit3 vflip,
                             bits6:4 priority (3b) */
    uint8_t  palette_off;
    uint8_t  reserved;
} nml_sprite_t;

/* --- Public API --- */
int  nml_open(void);              /* mmap and init */
void nml_close(void);
void nml_set_enable(int on);
void nml_write_sprite(int slot, const nml_sprite_t *s);
void nml_hide_sprite(int slot);     /* writes sentinel coords */
void nml_clear_sprites(void);
void nml_write_palette(int idx, uint8_t r, uint8_t g, uint8_t b);
void nml_write_tile(int col, int row, uint8_t tile_id);
void nml_set_player_state(int16_t px, int16_t py,
                          uint8_t hp, uint8_t wave, uint16_t level);
void nml_set_score(uint32_t score, uint32_t kills);
void nml_commit_frame(void);        /* sets SWAP, returns when committed */
int  nml_in_vblank(void);

#endif
```

6.4 Header: game.h

```
#ifndef GAME_H
#define GAME_H

#include <stdint.h>

#define MAX_ENTITIES   64    /* hard cap; sprite table caps render at 32 */
#define MAX_AUTOATK    8

typedef enum {
    ENT_NONE = 0,
    ENT_PLAYER,
    ENT_ENEMY_ARMED,         /* counts as combatant */
    ENT_ENEMY_UNARMED,       /* counts as non-combatant for end screen */
    ENT_PROJECTILE_PLAYER,
    ENT_PROJECTILE_AUTO,     /* mortar shell, gas puff, etc. */
    ENT_HAZARD_STATIC        /* barbed wire, gas cloud lingering */
} ent_kind_t;

typedef struct {
    ent_kind_t kind;
    int16_t x, y;            /* world position, pixels */
    int8_t  vx, vy;          /* per-frame velocity */
    uint8_t hp;
    uint8_t w, h;            /* AABB size */
    uint8_t sprite_id;
    uint8_t team;            /* 0 = player, 1 = enemy */
    uint16_t ttl;            /* frames until despawn (0 = persistent) */
    uint8_t  payload;        /* damage or auto-attack type */
```

```
} entity_t;

typedef enum {
    AA_MORTAR,
    AA_BARBED_WIRE,
    AA_GAS,
    AA_ARTILLERY,
    AA_COUNT
} autoatk_kind_t;

typedef struct {
    autoatk_kind_t kind;
    uint8_t  level;          /* 0 = not owned, 1..5 = upgraded */
    uint16_t cooldown;       /* frames remaining */
    uint16_t period;         /* frames between fires */
} autoatk_t;

typedef struct {
    entity_t  entities[MAX_ENTITIES];
    autoatk_t autoatks[AA_COUNT];
    uint16_t  wave;
    uint16_t  frame;
    uint32_t  score;         /* SW-maintained; written to NML_REG_SCORE each
        frame */
    uint32_t  kills_armed;
    uint32_t  kills_unarmed;
    uint8_t   player_level;
    uint16_t  xp;
    uint8_t   state;         /* MENU / PLAYING / LEVELUP / DEAD */
} game_t;

void game_init(game_t *g);
void game_tick(game_t *g, uint16_t input_bitmask);
void game_render(const game_t *g);   /* writes sprite table; must cull
                                        active entities down to <=32
                                        scored by: player > projectiles >
                                        nearby enemies > distant enemies
                                        > hazards. See render.c.
                                        Hidden entities use NML_SPRITE_HIDE. */

#endif
```

6.5 Main Loop

Fixed-step 60 Hz, tied to VBLANK via busy-poll on `STATUS.VBLANK`.

```
int main(void) {
    nml_open();
    input_start();               /* spawns reader thread (SNES or mouse) */
    game_t g;
    game_init(&g);
    nml_set_enable(1);

    while (g.state != STATE_QUIT) {
        struct timespec t0, t1;
        clock_gettime(CLOCK_MONOTONIC, &t0);

        uint16_t input = input_snapshot();
```

```
        game_tick(&g, input);
        game_render(&g);          /* writes sprites into shadow */
        nml_commit_frame();       /* SWAP at next VBLANK; blocks */

        clock_gettime(CLOCK_MONOTONIC, &t1);
        long us = (t1.tv_sec - t0.tv_sec) * 1000000L
                + (t1.tv_nsec - t0.tv_nsec) / 1000L;
        if (us > 16600) {         /* one frame at 60 Hz = 16.67 ms */
            fprintf(stderr, "frame overrun: %ld us\n", us);
        }
    }
    nml_close();
    return 0;
}
```

7 Hardware Architecture (SystemVerilog)

7.1 Module Hierarchy

```
nml_gpu                          (top-level peripheral)
|-- avalon_slave_iface           (Avalon-MM decode + register file)
|-- pll_25mhz                    (Quartus-generated PLL IP, 50 -> 25 MHz)
|-- vga_timing                   (HSYNC/VSYNC/x/y/blank generator, 640x480@60)
|-- sprite_table_ram             (M10K, 32 x 64b dual-port)
|-- tile_map_ram                 (M10K, 4800 x 8b dual-port)
|-- palette_ram                  (M10K, 256 x 24b dual-port)
|-- sprite_rom                   (M10K, 64 sprites x 256B, $readmemh)
|-- tile_rom                     (M10K, 64 tiles x 64B, $readmemh)
|-- sprite_eval                  (per-scanline sequential bbox FSM, fills active buf)
|-- compositor                   (tile + sprite + palette pipeline)
|   |-- line_buffer (x2)         (640 x 24b, double buffered)
|-- hud_overlay                  (HP bar, wave, score; reads player regs)
```

7.2 Top Module Interface

```
module nml_gpu (
    // Avalon-MM slave (Lightweight bridge, 50 MHz)
    input  logic        clk,
    input  logic        reset_n,
    input  logic [13:0] avs_address,    // 16 KB window, byte addr
    input  logic        avs_read,
    input  logic        avs_write,
    input  logic [31:0] avs_writedata,
    input  logic [3:0]  avs_byteenable,
    output logic [31:0] avs_readdata,
    output logic        avs_waitrequest, // tied 0; always single-cycle

    // VGA (driven by internal pix_clk derived from PLL)
    output logic [7:0]  vga_r,
    output logic [7:0]  vga_g,
    output logic [7:0]  vga_b,
    output logic        vga_hs,
    output logic        vga_vs,
```

```
    output logic        vga_blank_n,
    output logic        vga_sync_n,
    output logic        vga_clk
);
```

7.3 Submodule Interfaces (Abridged)

```
module vga_timing (
    input  logic        pix_clk, reset_n,
    output logic [9:0]  x,          // 0..799 (640 visible + porches)
    output logic [9:0]  y,          // 0..524 (480 visible + porches)
    output logic        visible,
    output logic        hsync, vsync,
    output logic        vblank, hblank
);

// sprite_eval: sequential FSM walks sprite table one entry per cycle
// during HBLANK. No parallel line_sprites bus -- eliminates 64x8 wiring.
module sprite_eval (
    input  logic        pix_clk, reset_n,
    input  logic [9:0]  next_scanline,    // y of upcoming line
    input  logic        eval_strobe,      // pulse at HBLANK start
    // Sprite table read port (one entry per cycle)
    output logic [4:0]  sprtab_raddr,
    input  logic [63:0] sprtab_rdata,
    // Output: up to 8 active sprites, registered at end of HBLANK
    output logic [7:0]  active_mask,
    output logic [63:0] active_sprites [0:7]  // latched, not a live bus
);

// compositor: receives latched active_sprites (not a wide combinational bus)
module compositor (
    input  logic        pix_clk, reset_n,
    input  logic [9:0]  x, y,
    input  logic        visible,
    input  logic [15:0] scroll_x, scroll_y,
    // Tile map read
    output logic [12:0] tilemap_raddr,
    input  logic [7:0]  tilemap_rdata,
    // Tile ROM read
    output logic [11:0] tilerom_raddr,
    input  logic [7:0]  tilerom_rdata,
    // Active sprite buffer (latched from sprite_eval, not a live fan-in)
    input  logic [7:0]  active_mask,
    input  logic [63:0] active_sprites [0:7],
    output logic [13:0] sprrom_raddr,
    input  logic [7:0]  sprrom_rdata,
    // Palette read
    output logic [7:0]  palette_raddr,
    input  logic [23:0] palette_rdata,
    // Pixel output
    output logic [23:0] rgb_out
);

module avalon_slave_iface (
    input  logic        clk, reset_n,
    input  logic [13:0] avs_address,
```

```
    input  logic         avs_read, avs_write,
    input  logic [31:0]  avs_writedata,
    input  logic [3:0]   avs_byteenable,
    output logic [31:0]  avs_readdata,
    // Register-file outputs to rest of design
    output logic         ctrl_enable,
    output logic         ctrl_swap_req,    // pulses 1 cycle on write
    output logic         ctrl_hud_on,
    output logic [31:0]  bg_scroll,
    input  logic         vblank_in,
    input  logic         swap_pending_in,
    // Memory write fan-out (one strobe per region)
    output logic         sprtab_we, palette_we, tilemap_we,
    output logic [12:0]  mem_waddr,
    output logic [31:0]  mem_wdata,
    // Memory read fan-in for SW reads
    input  logic [31:0]  sprtab_rdata_sw,
    input  logic [31:0]  palette_rdata_sw,
    input  logic [31:0]  tilemap_rdata_sw,
    // Player state out to HUD
    output logic [31:0]  player_pos, player_stats, score, kill_count
);
```

Key change from original design: the compositor no longer takes a wide `line_sprites[0:7]` combinational input bus (which would require $64 \times 8 = 512$ wires). Instead, `sprite_eval` latches its results into `active_sprites[0:7]` at the end of HBLANK; the compositor reads from this registered buffer during the visible scanline. This eliminates excessive routing and stays within DE1-SoC FPGA fabric constraints.

7.4 Sprite Eval FSM

The FSM runs during HBLANK, evaluating one sprite table entry per cycle:

```
// During HBLANK, evaluate which sprites are on the next scanline
always_ff @(posedge pix_clk) begin
    if (eval_strobe) begin
        sprite_idx   <= 0;
        active_count <= 0;
    end else if (sprite_idx < NUM_SPRITES) begin
        // Read sprtab_rdata (registered from previous cycle)
        if (next_scanline >= sprite_y &&
            next_scanline <  sprite_y + 16 &&
            sprite_x != HIDE_SENTINEL) begin
            if (active_count < 8)
                active_sprites[active_count] <= sprtab_rdata;
                active_count <= active_count + 1;
        end
        sprite_idx <= sprite_idx + 1;
        sprtab_raddr <= sprite_idx + 1;  // pre-fetch next entry
    end
end
// 32 sprites at 1 cycle each = 32 cycles; HBLANK is ~160 cycles at 25 MHz
```

7.5 Inter-Block Protocols

Most signals are plain wires of the widths given above, driven every cycle. Three interfaces have handshakes:

- **Avalon-MM slave:** `avs_waitrequest` tied 0. Writes commit on `avs_write` cycle; reads return combinationally same cycle. Every readable region is single-cycle accessible.
- **Sprite table swap:** SW writes shadow via `sprtab_we`, then `CTRL.SWAP=1`. Slave pulses `ctrl_swap_req` for one cycle; compositor latches a "swap pending" flag and flips active/shadow pointers on the next `vblank` rising edge. SW polls `STATUS.SWAP_PENDING` or waits for VBLANK IRQ.
- **Sprite eval → compositor:** `sprite_eval` drives `active_mask` and `active_sprites[0:7]` as registered outputs latched at end of HBLANK. Compositor samples them at start of visible scanline. No backpressure: 32 sequential bbox checks at 1 cycle each fit comfortably in the 160-cycle HBLANK window at 25 MHz.

8 File Formats

The project reads two files at FPGA build time (sprite ROM, tile ROM) and one at program startup (wave table). No files are read during gameplay.

8.1 `sprite_rom.hex` – Sprite ROM

Intel-hex for `$readmemh`, one byte per line, no addresses. Exactly $64 \times 256 = 16384$ bytes. Each 256-byte block is one 16×16 sprite; pixel (u, v) of sprite S is at byte $S \cdot 256 + v \cdot 16 + u$. Byte value is a palette index; `0x00` = transparent.

8.2 `tile_rom.hex` – Tile ROM

Same format, $64 \times 64 = 4096$ bytes. Each 64-byte block is one 8×8 tile in row-major order. No transparency. Tile pixels are used to pre-fill the line buffer between sprite draws (replacing the previous approach of clearing to zero).

8.3 `waves.dat` – Wave Table

ASCII, one wave per line, whitespace-separated. Loaded once at startup into a static array; never modified at runtime.

```
# wave_num duration_sec spawn_period_frames armed_count unarmed_count enemy_hp
1   30   60   8    0   2
2   30   45  10    0   2
3   45   45  12    1   3
...
20  60   15  20   18   6
```

Lines starting with `#` are comments. The narrative arc (rising `unarmed_count`) lives entirely in this file.

9 Risk Register Update

Refinements from this design; the proposal's risk table still applies.

- **Input device:** SNES controller is the primary input. USB mouse is the fallback. Keyboard input is not permitted per professor review. SNES HID adapter compatibility to be verified on DE1-SoC USB host by week 1.
- **Per-scanline sprite cap 8**, enforced by `sprite_eval` priority-culling. The 32-entry table is a frame-level limit; only 8 highest-priority sprites render per line.
- **Frame-level cap 32**, enforced in SW by `game_render()`. Late waves produce 40+ enemies; without culling, overflow entries would silently not render and gameplay would desync from visuals.
- **No active bit:** sprites are hidden by writing sentinel coordinates ($x = y = -256$). The sprite eval FSM skips them on bounding-box check. This eliminates one field from the sprite table entry and simplifies hardware.
- **Wiring reduction:** compositor no longer uses a parallel `line_sprites[0:7]` fan-in bus. Sequential sprite eval with a registered active buffer reduces inter-module wiring from $64 \times 8 = 512$ to $64 \times 8 = 512$ latched bits (registered, not combinational), which the router handles without congestion.
- **VBLANK synchronization:** sprite table uses VBLANK-based double-buffering. Shadow memory is not used for synchronization. SW writes to shadow, sets `CTRL.SWAP`, and the swap commits at VBLANK.
- **Line buffer clear:** the new fill buffer is pre-loaded with tile background pixels rather than zeroes before sprite writes, so no black flash appears between frames.
- **PLL frequency:** Cyclone V PLL cannot produce exactly 25.175 MHz from 50 MHz; 25 MHz is used instead. Monitors tolerate the deviation; verify on physical VGA by week 2.
- **Sprite ROM port contention:** line-buffer architecture (Section 4.3) uses one sequential fetch stream, so one M10K dual-port ROM suffices. No replication needed.
- **SDL simulator:** game logic is developed and tested against the SDL simulator before FPGA integration. ROM data is compiled in as static arrays; no runtime file I/O.
- **Frame-overrun watchdog:** `main.c` logs frames >16.6 ms. Dev aid only; shipping builds silently drop overruns (VBLANK wait absorbs slack).
- **Background scrolling decision is final:** the arena is single-screen in baseline. No phased introduction of scrolling; the design either implements it fully or omits it entirely.

10 Milestones (Unchanged from Proposal)

Week	Milestone	Owner(s)
1–2	VGA tile rendering; SNES/mouse input; basic sprite display; SDL simulator	All
3–4	Player movement and shooting; enemy spawning, chase AI; SW collision	Rohit, Kambi
5	Auto-attacks from SW; item selection UI; narrative wave progression	Nicola, Rohit
6	Integration testing, visual polish, bug fixes, demo prep	All